ICMC USP SÃO CARLOS

USP Universidade de São Paulo



IA, Otimização e IoT para Indústrias e Negócios
Do Zero ao MLOps

PROFESSOR

Dr. Cláudio Fabiano Motta Toledo

OpAIoT
Optimization • AI • IoT

Módulo 1

Fundamentos de Dados e Aprendizado de Máquina

Aplicado a Operações e Negócios

OpAIoT
Optimization • AI • IoT

Universidade de São Paulo

Transformação Numérico Numérico

Sumário

01 O que é Transformação Numérico Numérico?

02 Padronização

03 Normalização Min-Max

04 Escalonamento Robusto

05 Transformações de potência e log

06 Resumo

O que é Transformação Numérico - Numérico ?

Definição: processo de modificar valores numéricos existentes em outros valores numéricos para melhorar o desempenho de algoritmos de aprendizado de máquina.

Por que isso é tão importante?

- **Escala comparáveis:** muitos algoritmos funcionam melhor quando os atributos estão em escalas comparáveis ou têm distribuições menos problemáticas
  - **Convergência mais rápida:** algoritmos como gradiente descendente convergem mais rápido quando os recursos estão em escalas semelhantes
  - **Prevenção de viés**: evita que variáveis com magnitudes maiores dominem o modelo.



Há situações em que essa variação deve ser preservada por ser importante para a indução de um bom modelo.

Tipos de Transformação Numérico - Numérico

- Padronização
- Normalização Min-Max
- Escalonamento Robusto
- Transformações de potência e log

Transforma os dados para média 0 e desvio-padrão 1: $x_{ij}^{novo} = \frac{x_{ij} - \mu_j}{\sigma_j}$

Quando faz sentido?

Ao usar **modelos sensíveis à escala,** como: regressão com regularização, SVM, KNN e redes neurais.

```
from sklearn.preprocessing import StandardScaler
import numpy as np

X = np.array([[20], [30], [40]])
scaler = StandardScaler()
X_pad = scaler.fit_transform(X)
print(X_pad)
```



Valores passam a representar quão longe da média cada valor está, em unidades de desvio-padrão.

Reescala os valores para um intervalo fixo, geralmente [0,1], preservando a ordem relativa dos dados, mas muda a escala.

Quando faz sentido?

Quando você quer que todas as variáveis fiquem no mesmo intervalo.

$$x^{novo} = min^{novo} + \frac{x - min(x)}{max(x) - min(x)} \left(max^{novo} - min^{novo}\right)$$

min^{novo} : novo valor mínimo definido para o atributo

max^{novo} : novo valor máximo definido para o atributo

$min_i(x)$: valor mínimo atual do atributo

$max_i(x)$: valor máximo atual do atributo

```python
from sklearn.preprocessing import MinMaxScaler
import numpy as np

X = np.array([[10], [20], [30]])
scaler = MinMaxScaler()
X_mm = scaler.fit_transform(X)
print(X_mm)
```



Não reduz o efeito de outliers; apenas os reescala.

Parecido com a padronização, mas usa medidas menos sensíveis a outliers: a mediana e o intervalo interquartil (IQR).

Quando faz sentido?

Há muitos valores extremos e você não quer que eles dominem a escala.

$$x^{novo} = \frac{x - mediana}{IQR} = \frac{x - mediana}{Q_3 - Q_1}$$

$$x^{novo} = \frac{x - mediana}{Q_{q_{max}} - Q_{q_{min}}}$$

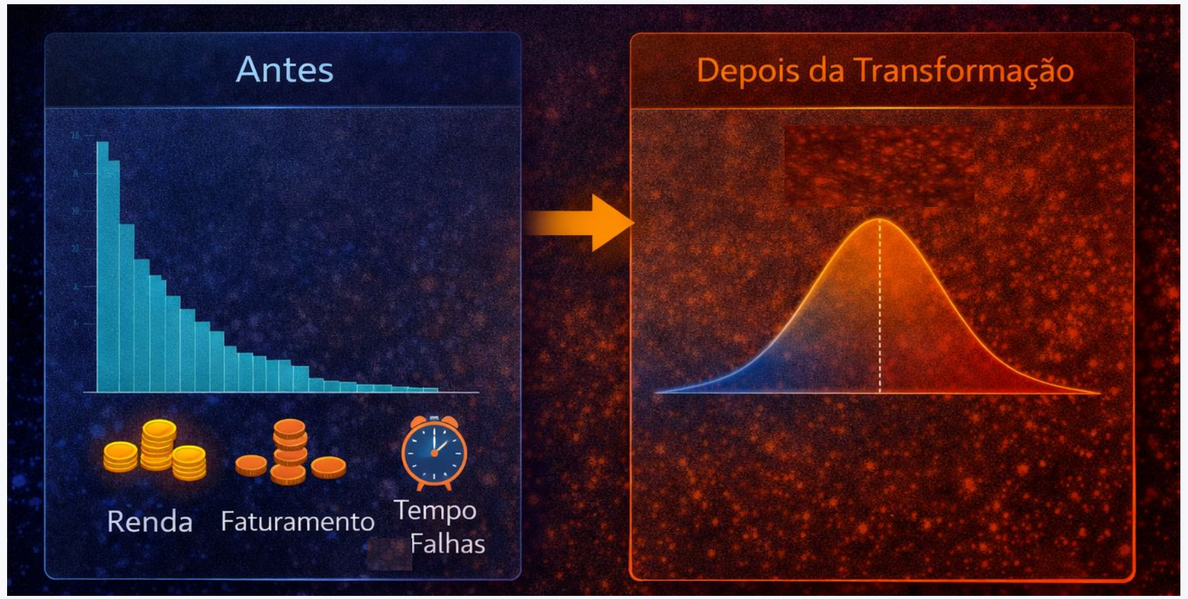
```
from sklearn.preprocessing import RobustScaler
import numpy as np

X = np.array([[2000], [2200], [2500], [2600], [50000]])
scaler = RobustScaler()
X_rb = scaler.fit_transform(X)
print(X_rb)
```

Tornam os dados mais parecidos com uma distribuição normal.

Quando faz sentido?

Dados como renda, faturamento e tempo entre falhas costumam ser **muito concentrados** em valores pequenos e ter poucos valores muito altos. Nesses casos, transformações de potência ajudam a tornar os dados “mais parecidos com uma distribuição normal”



- **Transformação de Box-Cox**
- **Transformação de Yeo-Johnson**

- **Transformação de Box-Cox:** definida apenas para dados positivos (y>0).

$$y_i^{(\lambda)} = \begin{cases} \frac{y_i^{\lambda}-1}{\lambda}, & \text{se } \lambda \neq 0 \\ \ln(y_i), & \text{se } \lambda = 0 \end{cases}$$

- Se houver zeros ou negativos, não pode ser aplicada diretamente.
- **O que faz:** “comprime” os valores grandes, reduzindo a assimetria.
- **Objetivo:** tornar a distribuição o mais simétrica possível (próxima da normal).

- **Transformação de Yeo-Johnson:** extensão da Box-Cox para trabalhar com dados positivos, negativos e zeros.

$$y_i^{(\lambda)} = \begin{cases} \frac{(y_i+1)^{\lambda}-1}{\lambda} & \text{se } \lambda \neq 0,\ y_i \geq 0 \\ \ln(y_i+1) & \text{se } \lambda = 0,\ y_i \geq 0 \\ -\frac{(1-y_i)^{2-\lambda}-1}{2-\lambda} & \text{se } \lambda \neq 2,\ y_i < 0 \\ -\ln(1-y_i) & \text{se } \lambda = 2,\ y_i < 0 \end{cases}$$

- **Mais flexível:** ideal para datasets com valores negativos como retornos financeiros, temperaturas em Celsius, etc.

Quando você deve pensar em transformação numérico-numérico?

- Se os atributos estão em escalas muito diferentes

  ➔ pense em padronização ou Min-Max.

- Se há outliers importantes

  ➔ pense em RobustScaler.

- Se a variável está muito assimétrica

  ➔ pense em log, Box-Cox ou Yeo-Johnson.

OpAIoT
Optimization • AI • IoT

Universidade de São Paulo

Referências

- Faceli, Katti, et al. "Inteligência artificial: uma abordagem de aprendizado de máquina." (2021).
- Pandas User Guide. Discretization and quantiling. https://pandas.pydata.org/pandas-docs/stable/user_guide/basics.html
- sklearn.preprocessing https://scikit-learn.org/stable/api/sklearn.preprocessing.html#





Obrigado!

opaiot@icmc.usp.br
+55 (16) 99962-7637

PROFESSOR

Dr. Cláudio Fabiano Motta Toledo

OpAIoT
Optimization • AI • IoT